

Plan de Desarrollo de Software: Corazón Contento

(Versión 0.0.2)

Equipo Corazón Contento

4 de octubre de 2025

Índice

1. Objetivos SMART, Análisis de Requisitos y Recursos	2
2. Herramientas y Tecnologías	2
2.1. Stack de Desarrollo de Aplicación	2
2.2. Stack de Gestión y DevOps	2
3. Ciclo de Vida de Desarrollo y Flujo de Trabajo (DevOps)	2
3.1. Taiga: Gestión Ágil del Proyecto	2
3.2. GitHub: Control de Versiones y CI/CD	3
3.3. SonarCloud: Auditoría de Calidad de Código	3
3.4. Ansible: Despliegue Automatizado	3
3.5. Slack: Comunicación y Notificaciones Centralizadas	3
4. Ejemplo Práctico: Implementando la HU HP-001	3

1. Objetivos SMART, Análisis de Requisitos y Recursos

Esta sección se mantiene como la definida en el documento inicial `corazonContentoV0.0.1.pdf`. Los objetivos del proyecto, así como los requisitos funcionales y no funcionales, están claramente establecidos y sirven como base para el desarrollo que se describe a continuación.

2. Herramientas y Tecnologías

Para la construcción, gestión y despliegue de la aplicación “Corazón Contento”, se ha definido un stack tecnológico moderno que abarca tanto el desarrollo del software como la cultura DevOps para su entrega continua.

2.1. Stack de Desarrollo de Aplicación

- **Backend:** Python
- **Frontend:** React y React Native
- **Base de Datos:** PostgreSQL
- **Infraestructura:** Google Cloud Platform (GCP)

2.2. Stack de Gestión y DevOps

- **Comunicación:** Slack
- **Gestión Ágil de Proyecto:** Taiga.io
- **Control de Versiones e Integración Continua (CI/CD):** GitHub y GitHub Actions
- **Despliegue y Configuración Automatizada:** Ansible
- **Análisis Estático y Calidad de Código:** SonarCloud

3. Ciclo de Vida de Desarrollo y Flujo de Trabajo (DevOps)

El equipo Corazón Contento.ªadoptará un flujo de trabajo ágil basado en Scrum, soportado por un pipeline de Integración y Despliegue Continuo (CI/CD) para asegurar entregas rápidas y de alta calidad.

El flujo general es el siguiente:

Planificación → Desarrollo → Auditoría de Calidad → Integración Continua → Despliegue Automatizado → Notificación

A continuación, se detalla el uso específico de cada herramienta dentro de este ciclo.

3.1. Taiga: Gestión Ágil del Proyecto

Taiga será nuestro centro de mando para la planificación y seguimiento del trabajo.

- **Product Backlog:** El Product Owner cargará todas las Historias de Usuario (HU) definidas (ej. **HP-001** a **HP-005** y **HC-001** a **HC-005**) en el backlog.
- **Sprints:** El trabajo se organizará en Sprints de 2 semanas. En la reunión de *Sprint Planning*, el equipo seleccionará HUs del backlog y las descompondrá en tareas técnicas.
- **Tablero Kanban:** El progreso de cada tarea se visualizará en el tablero Kanban, moviendo las tarjetas por las columnas: *To Do*, *In Progress*, *In Review*, *Done*.
- **Seguimiento:** Se utilizarán los *Burndown charts* de Taiga para monitorear el progreso del Sprint.

3.2. GitHub: Control de Versiones y CI/CD

GitHub será el repositorio central de nuestro código y el motor de nuestro pipeline de automatización.

- **Estrategia de Ramas:**
 - **main:** Contendrá el código de producción, estable y desplegado.
 - **develop:** Rama de integración donde se consolidan todas las nuevas funcionalidades.
 - **feature/<clave_historia>:** Cada funcionalidad se desarrollará en su propia rama (ej. `feature/HP-001`).
- **Pull Requests (PRs):** Al terminar una funcionalidad, se abrirá un Pull Request de la rama `feature/...` hacia `develop`, lo que disparará el pipeline.
- **GitHub Actions (CI/CD):** Se configurarán flujos de trabajo para automatizar la compilación, pruebas unitarias, análisis de calidad y el despliegue.

3.3. SonarCloud: Auditoría de Calidad de Código

Para asegurar que solo código limpio y seguro llegue a producción, SonarCloud analizará cada Pull Request.

- **Quality Gate:** Se establecerá un "portal de calidad" con criterios mínimos (ej. cobertura >80 %, cero bugs críticos).
- **Integración con GitHub:** Si un PR no cumple con el Quality Gate, SonarCloud lo marcará como "fallido" en GitHub, bloqueando la fusión hasta que se corrija.
- **Notificación en Slack:** Se configurará un webhook para notificar en el canal `#desarrollo` los resultados del análisis.

3.4. Ansible: Despliegue Automatizado

Ansible eliminará los despliegues manuales, haciéndolos repetibles y libres de errores.

- **Playbooks:** Se crearán "recetarios" (playbooks) para automatizar la configuración de la infraestructura (`provision-server.yml`) y el despliegue de la aplicación (`deploy-backend.yml`).
- **Entornos:** Se gestionarán al menos dos entornos: **Staging** (para pruebas) y **Producción** (para usuarios finales).

3.5. Slack: Comunicación y Notificaciones Centralizadas

Slack será el pegamento que una todas las herramientas, manteniendo al equipo informado en tiempo real.

- **Canales:**
 - **#desarrollo:** Notificaciones de GitHub y SonarCloud.
 - **#despliegues:** Notificaciones de GitHub Actions y Ansible sobre el estado de los despliegues.
 - **#taiga:** Notificaciones sobre nuevas historias o cambios en el sprint.

4. Ejemplo Práctico: Implementando la HU HP-001

Para simular el flujo, veamos cómo se implementaría la historia **HP-001: "Publicar alimentos sobrantes"**.

1. **Planificación (Taiga):** La HU se mueve al Sprint actual y se descompone en tareas técnicas.
2. **Desarrollo (GitHub):** Un desarrollador crea la rama `feature/HP-001` y comienza a programar.
3. **Integración (GitHub/SonarCloud):** Al terminar, abre un Pull Request a `develop`. GitHub Actions inicia el pipeline, ejecutando pruebas y el análisis de SonarCloud.

4. **Revisión y Fusión:** Otro miembro del equipo revisa el código. Al ver que todas las comprobaciones son exitosas, aprueba y fusiona el PR.
5. **Despliegue (Ansible):** La fusión a `develop` dispara un workflow que ejecuta el playbook de Ansible para desplegar la nueva versión en el entorno de **Staging**.
6. **Notificación (Slack):** El canal `#despliegues` recibe un mensaje: *"Despliegue de 'feature/HP-001' en STAGING finalizado con éxito "*.
7. **Validación:** El equipo de QA prueba la funcionalidad en Staging antes de planificar el pase a producción.